Module 9
Assignment 9A: Contact List App

Course: CS 85 - PHP Programming
Instructor: Vicky Seno

Project Overview

For this assignment, I created a Laravel Contact List application connected to a MySQL database. The application follows the MVC pattern and supports all required CRUD operations for managing contacts.
The standard version of the application is available locally at:
http://127.0.0.1:8000/contacts
I also created an additional Advanced Workbench that demonstrates extended Laravel and database features beyond the basic assignment requirements.
Required CRUD Features
The standard Contact List App allows a user to:

- Add a contact with a required name, email address, and phone number.
- View all contacts in a contact list.
- Open a separate form to edit an existing contact.
- Update the contact information.
- Delete a contact from the database.
- Receive confirmation messages after successful operations.

The contact list displays the required fields:

- Name
- Email
- Phone

Each contact also has Edit and Delete actions.

Project Links

- Module 9A Documentation https://github.com/sergehall/cs85-php-programming/blob/main/assignments/module9a/README.md

- Complete GitHub Repository https://github.com/sergehall/cs85-php-programming

- Laravel Routes https://github.com/sergehall/cs85-php-programming/blob/main/routes/web.php

- Contact Model https://github.com/sergehall/cs85-php-programming/blob/main/app/Models/Contact.php

Required Contact List App

Local URL: http://127.0.0.1:8000/contacts

- Resource Controller https://github.com/sergehall/cs85-php-programming/blob/main/app/Http/Controllers/ContactController.php

- Index, Create, and Edit Blade Views https://github.com/sergehall/cs85-php-programming/tree/main/resources/views/contacts

- Form Request Validation https://github.com/sergehall/cs85-php-programming/tree/main/app/Http/Requests

- CRUD Feature Tests https://github.com/sergehall/cs85-php-programming/blob/main/tests/Feature/ContactResourceTest.php

Advanced CRUD Workbench

Local URL: http://127.0.0.1:8000/assignments/module9a/contacts

- Advanced Workbench Interface https://github.com/sergehall/cs85-php-programming/blob/main/resources/views/assignments/module9a/contacts.blade.php

- Advanced Workbench Controller https://github.com/sergehall/cs85-php-programming/blob/main/app/Http/Controllers/Assignments/Module9aContactController.php

- JSON Dataset Controller https://github.com/sergehall/cs85-php-programming/blob/main/app/Http/Controllers/Assignments/Module9aContactDatasetController.php

- Advanced Application Services https://github.com/sergehall/cs85-php-programming/tree/main/app/Services/Modules/Module9A

- Advanced Feature Tests https://github.com/sergehall/cs85-php-programming/blob/main/tests/Feature/Module9aContactListTest.php

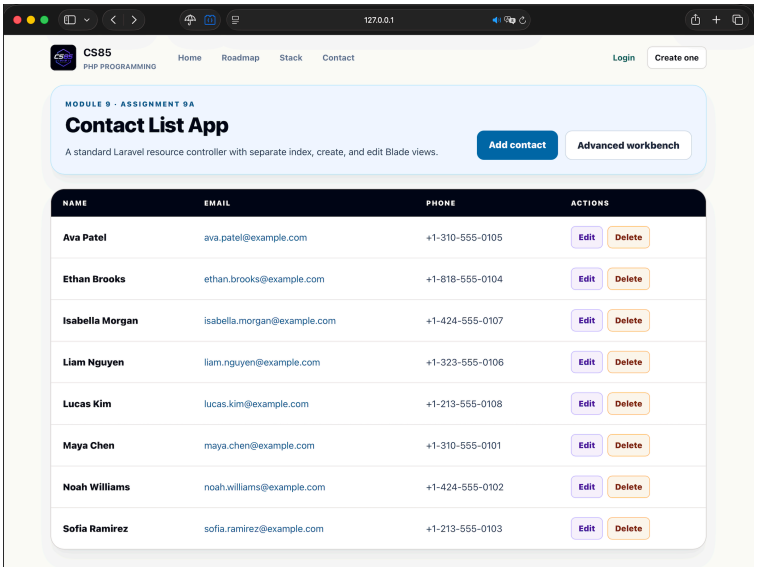

Figure 1. The standard Contact List App displays contacts from the MySQL database and provides Add, Edit, and Delete actions.

Laravel Concepts Used

Routing
I used Laravel's standard resource routing:
Route::resource('contacts', ContactController::class);
This resource route provides the standard CRUD endpoints for listing, creating, storing, viewing, editing, updating, and deleting contacts.

Controller
The ContactController is implemented as a Laravel resource controller and contains:

- index() — displays the contact list.
- create() — displays the Add Contact form.
- store() — validates and saves a new contact.
- show() — retrieves an individual contact.
- edit() — displays the Edit Contact form.
- update() — validates and updates a contact.
- destroy() — deletes a contact.

Model
The Contact Eloquent model communicates with the contacts table in MySQL. Its fillable fields include the required contact information:

- name
- email
- phone

Migrations and MySQL
The application is connected to MySQL. Laravel migrations create and maintain the database schema.
The contacts table includes:

- A primary key.
- A required name.
- A required and unique email address.
- A required phone number.
- Laravel timestamps.

Blade Views
The standard application uses separate Blade views as required:

- index.blade.php — displays the contact list.
- create.blade.php — displays the Add Contact form.
- edit.blade.php — displays the Edit Contact form.
- A shared form partial is used to avoid duplicating the form fields.

Validation
Laravel Form Request classes validate submitted data before it is stored in the database.
The validation rules ensure that:

- Name is required.
- Email is required and must have a valid format.
- Email addresses must be unique.
- Phone number is required.
- Email addresses are normalized before being saved.

Validation errors are displayed next to the corresponding form fields.
CSRF Protection

All forms that modify data include Laravel CSRF tokens. Update and delete forms use Laravel method spoofing for the PUT and DELETE HTTP methods.

Advanced Workbench

In addition to the required Contact List App, I created an advanced CRUD workbench at:
http://127.0.0.1:8000/assignments/module9a/contacts
This interface demonstrates additional Laravel features, including:

- Contact statistics and database record counts.
- GET filters and global contact search.
- Filtering by name, email, phone, group, role, and status.
- Sorting and pagination.
- JSON dataset import.
- Idempotent database upserts.
- Contact groups with a one-to-many relationship.
- Eloquent ORM queries and relationships.
- Form Request validation.
- Database transactions.
- Create, Read, Update, and Delete operations from one interface.
- A raw JSON response endpoint.
- Route model binding.
- Rate limiting and additional write-operation protection.

The advanced workbench uses the HTTP methods GET, POST, PUT, and DELETE and demonstrates how Laravel connects routes, controllers, validation, Eloquent models, Blade views, and MySQL.

Testing and Code Quality

The application was tested with Laravel automated tests.
Final verification results:

- 171 automated tests passed.
- 1,472 assertions passed.
- Laravel Pint formatting checks passed.
- PHPStan static analysis passed.
- The frontend production build completed successfully.

The tests cover the standard resource routes, creating contacts, validation, duplicate email prevention, editing, updating, deleting, CSRF-protected forms, and the advanced workbench operations.

Advanced Workbench in your browser: http://127.0.0.1:8000/assignments/module9a/contacts

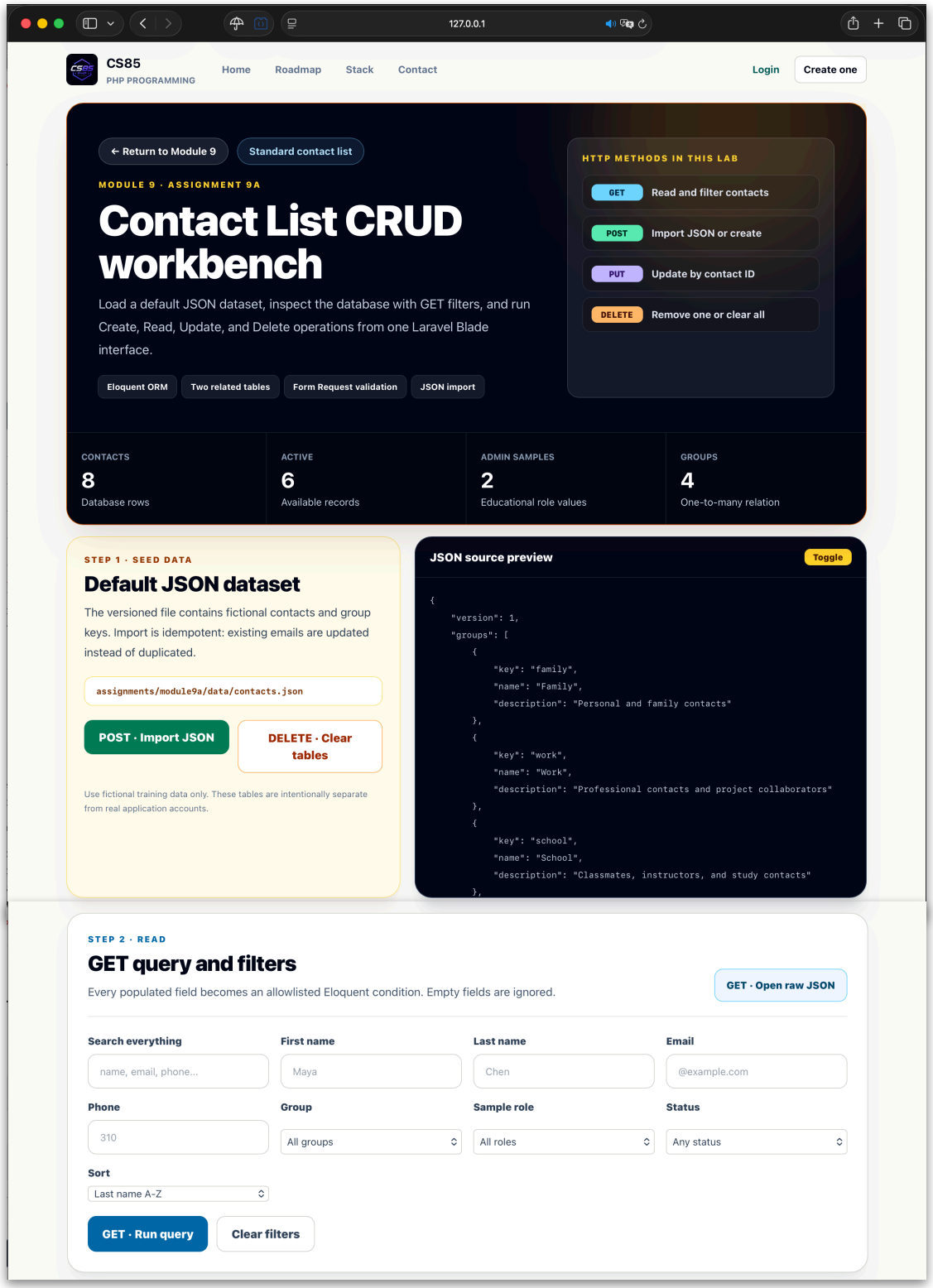

Figure 2.1. The Advanced Contact List CRUD Workbench shows database statistics, supported HTTP methods, filtering tools, JSON import, relationships, and extended CRUD functionality.

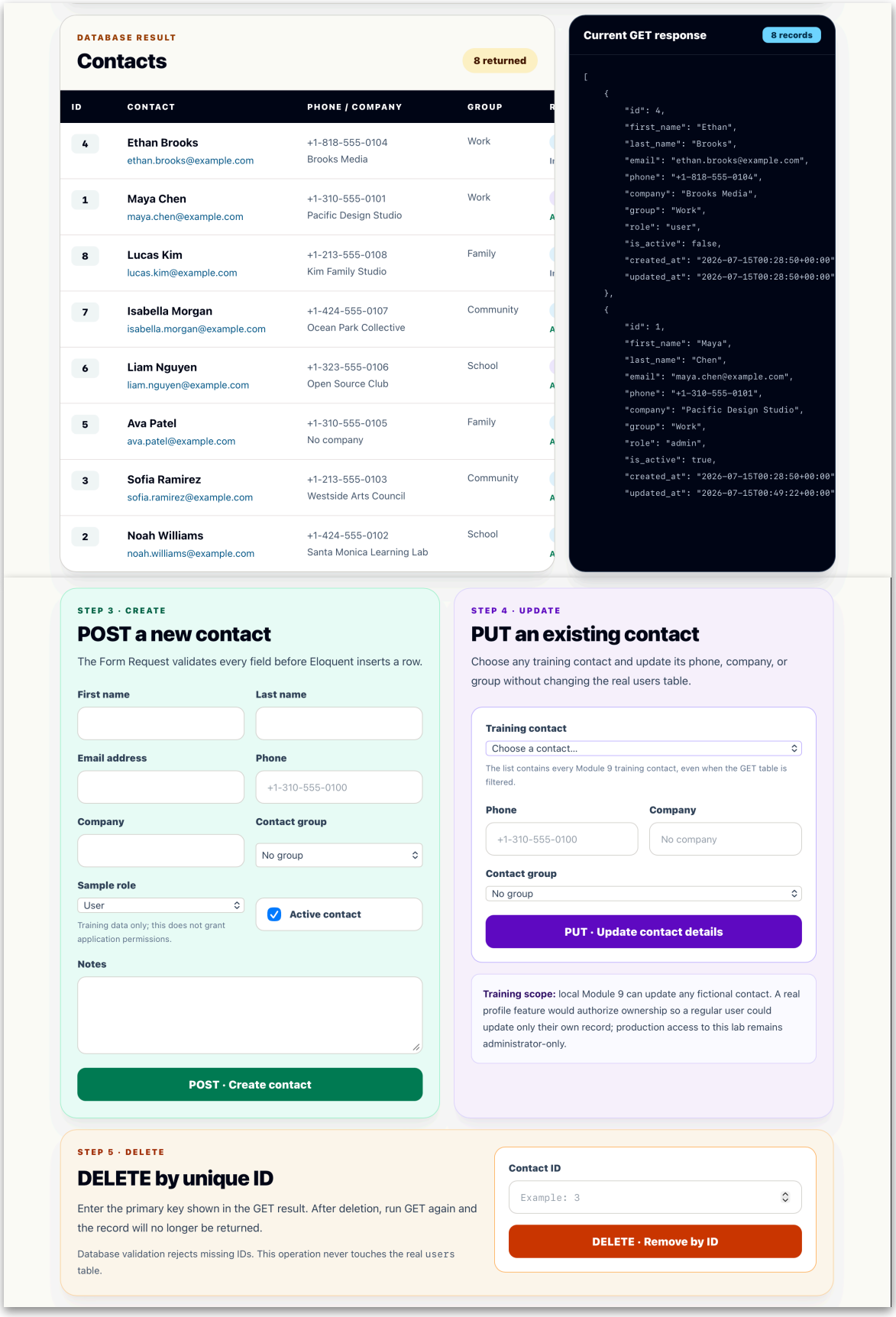

Figure 2.2. The Advanced Contact List CRUD Workbench shows database statistics, supported HTTP methods, filtering tools, JSON import, relationships, and extended CRUD functionality.

Testing and Code Quality

The application was tested with Laravel automated tests.

Final verification results:

- 171 automated tests passed.
- 1,472 assertions passed.
- Laravel Pint formatting checks passed.
- PHPStan static analysis passed.
- The frontend production build completed successfully.

The tests cover the standard resource routes, creating contacts, validation, duplicate email prevention, editing, updating, deleting, CSRF-protected forms, and the advanced workbench operations.

Conclusion

The project fulfills the requirements of Module 9 Assignment 9A by providing a complete Laravel CRUD application connected to MySQL and built with models, views, controllers, migrations, validation, Blade templates, resource routes, and CSRF protection.

The standard Contact List App directly follows the assignment instructions, while the Advanced Workbench extends the project with additional database, filtering, relationship, JSON, testing, and security features.